Ders 5 Genişletilmiş Notu: Iterator, Memento ve Observer Kalıpları

Ders 5 Genişletilmiş Notu: Iterator, Memento ve Observer Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Iterator Kalıbı
- 2. Iterator Gerçekleme Kararları
- 3. Iterator Örneği: Birleşen İki Lokantanın Menüsü
- 4. Memento Kalıbı
- 5. Memento'da Veri Gizliliğinin Gerçeklenmesi
- 6. Memento Örneği: Geri Alma ve Yineleme
- 7. Observer Kalıbı
- 8. Observer Gerçekleme Ayrıntıları
- 9. Observer Örneği: Müşteri İlişkileri Programı
- 10. Birbirine Benzeyen Kalıplar Üzerine
- 11. Örnek Sorular

1. Iterator Kalıbı

Geçen hafta Command kalıbıyla kalınmıştı. Ders, davranışsal (behavioral) kalıplardan Iterator ile devam etti. Iterator, Türkçeye "iterasyon" diye yamuk bir biçimde geçmiş bir sözcüktür. Tekrarlama ve bir sıraya göre tek tek ele alma anlamlarına gelir.

1.1 Amaç ve sorun

Elimizde bir parça-bütün yapısı vardır: küçük parçalardan oluşan bir bütün. Bu bütünle ilgili bir işlemi, bütünü oluşturan bütün parçaları dolaşarak ya da bütün parçaları aynı işleme tabi tutarak yapmak isteriz. Bunu yaparken parçaların iç yapısını açığa çıkarmamaya çalışırız. Slayttaki tanım şöyledir (ekrandan, 00:10:00):

- Amaç: parçalardan oluşan bir bileşenin parçalarını, bileşenlerin iç yapısını açığa çıkarmadan, istenilen bir biçimde dolaşmak.
- Örnek: bir veri yapısı içerisinde dolaşmak.
- Sorun: veri yapısının arayüzünü birçok farklı dolaşım türü ile şişirmek istemeyiz.

En basit örnek bir veri yapısında dolaşmaktır. Listeler gibi tek boyutlu, birbirini izleyen düğümlerden oluşan doğrusal yapılarda bile birçok varyasyon vardır. Normal bir listenin başı ve sonu vardır ve tek yönlü dolaşılır. Çift yönlü bağlı listelerde her düğüm yalnız sonraki düğümün değil, önceki düğümün de işaretçisini tutar. İlk düğümün önceki (previous) işaretçisi ve son düğümün sonraki (next) işaretçisi boştur. Dairesel (circular) listelerde son düğümün next işaretçisi baş düğümü gösterir. Çift yönlüyse baş düğümün previous işaretçisi de null değil, son düğümü gösterir. Yalnız listede tek yönlü/çift yönlü ve dairesel/değil olmak üzere iki kere iki, dört varyasyon vardır.

Ağaçlarda da durum benzerdir. İkili ağaçlarda (binary tree) her ara düğümün en fazla iki çocuğu olabilir; n çocuğu olabiliyorsa n-li (n-ary) ağaç denir. Normalde çocuk düğüm ebeveynine işaretçi tutmaz, ama tutan türleri de vardır. Yaprak düğümlerin kök düğümü gösterebildiği türler de vardır. Bunların dışında indekslemede kullanılan farklı ağaç yapıları, tablo ve map yapıları da vardır. Yapı nasıl olursa olsun, istenen şey bütün elemanları dolaşıp hepsine bir soru sormak ya da hepsine bir iş yaptırmaktır.

1.2 Aynı yapıda farklı dolaşım algoritmaları

Bir ikili ağacı dolaşmanın da farklı algoritmaları vardır. Hoca ekrana küçük bir tam ikili ağaç çizdi: kökün iki çocuğu, onların da ikişer çocuğu var. Sonra LDR (Left-Data-Right) dolaşımını adım adım gösterdi:

1. Kökten başlanır. Önce gidilebildiği kadar sola gidilir; bu düğümlerde ne yazdığına henüz bakılmaz.
2. Kökün sol çocuğuna, oradan da onun sol çocuğuna gidilir. Bu düğümün sol işaretçisi yoktur; düğümdeki veri ele alınır: 3.
3. Bu düğümün sağı da yoktur. Bir üste dönülür. Orada sol yapılmıştı; şimdi veri ele alınır: 5.
4. Bu düğümün sağ çocuğu vardır; ona gidilir. Solu yoktur, veri alınır: 6. Sağı yoktur, geri dönülür.
5. Köke dönülür. Kökte sol yapılmıştı; kökün verisi alınır: 9. Sonra sağ alt ağaca geçilmek zorundadır.

Örnekte ağaç bir sıralı indeks ağacı gibi düşünülmüştür; bu yüzden LDR sırası 3, 5, 6, 9, … biçiminde artan sırada ilerler. Hoca sağ alt ağacın değerlerini vermedi. Geri dönüşler, düğümde ebeveyn işaretçisi varsa onunla, yoksa gidilen düğümleri bir yığına (stack) push edip sonra pop ederek yapılabilir.

LDR tek algoritma değildir. RDL (right, data, left), DLR (data, left, right) ve DRL (data, right, left) de ayrı algoritmalardır. Bu karmaşıklıktan uzak bir biçimde parça-bütün ilişkisindeki bütün varlıklarda dolaşmak istenir. Ama veri yapısının arayüzü bu farklı dolaşım yöntemleriyle şişirilmek istenmez. Ayrıca listelerde, ağaçlarda ve map'lerde aynı biçimde çalışabilecek eş biçimli bir dolaşım altyapısı istenir.

1.3 Java'nın Iterator'u ve KISS

Hoca bir parantez açtı (ekrandan, 00:10:00). Java'da herhangi bir Collection'dan, kendisini gezmek üzere bir Iterator nesnesi istenebilir. Ancak bu nesne Iterator tasarım kalıbına tam olarak uymaz; kalıbın basitleştirilmiş bir hâlidir. Basit demek kötü demek değildir: çoğu durumda Java'nın Iterator'u da işe yarar.

> **Hocanın vurgusu:** KISS ilkesi: "Keep it simple, stupid", yani işi basit tut, tutmazsan aptallık etmiş olursun. Java'nın Iterator'u işinize yarıyorsa onu kullanın. Yaramıyorsa ya da Iterator altyapısı olmayan bir dille çalışıyorsanız bu kalıbı kullanabilirsiniz.

1.4 Birinci çözüm: List ve ListIterator

Dörtlü Çete'nin (Gang of Four) önerdiği birinci çözümün şeması şöyledir (ekrandan, 00:15:00):

```
List                                ListIterator
------------------   list           ---------------
Count()             <------------   First()
Append(Element)                     Next()
Remove(Element)                     IsDone()
...                                 CurrentItem()
                                    ---------------
                                    index
```

`ListIterator`'a bir liste verilir; aralarında birebir bir sahiplik vardır. Şemada ilişkinin `list` adıyla yazılması, `ListIterator`'da `List` türünden `list` adlı bir üye alan olduğu anlamına gelir. Listenin metotları: `Count()` eleman sayısını verir, `Append()` eleman ekler, `Remove()` eleman siler. Iterator'un metotları: `First()` ilk elemana erişir, `Next()` sonraki elemana geçer, `IsDone()`

bütün elemanların dolaşılıp dolaşılmadığını söyler, `CurrentItem()` o anda bakılan elemanı verir. Tamsayı `index` alanı kaçıncı elemanda olunduğunu tutar.

Çözümün güçlü yönü: veri yapısı ile dolaşım biçimi birbirinden soyutlanmıştır. Kaçınılmak istenen dolaşım metotları veri yapısından ayrılmış, böylece uyum (cohesion) artmıştır. Liste yalnız kendi içeriğiyle ilgili işlemleri yapar. Dolaşmak ayrı bir sorumluluk olarak ele alınır ve "değişeni bul, sarmala" ilkesi doğrultusunda başka bir sınıfa verilir.

Çözümün zayıf yönü: belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşmuştur. Bu bağlılığı azaltmak için genel bir liste yapısı arayüzü ve genel bir dolaşım biçimi arayüzü oluşturulabilir (ekrandan, 00:15:00).

Hoca burada sesli düşündü. Farklı liste türleri ve onlara uyan farklı iteratorlar olacağı için önce Abstract Factory akla gelebilir. Ama bu ailelerin bireyleri eş biçimli değildir: listenin dairesel, çift bağlı gibi üyeleri vardır, ağacın böyle üyeleri yoktur. Bu yüzden hoca bu fikirden vazgeçti. Factory Method ise işe yarar: bir veri yapısından kendisini dolaşacak bir iterator istenir. Bu bağımlılığı biraz azaltır, ama iterator yine belli bir somut veri yapısıyla ilişkili olmak zorundadır.

1.5 İkinci çözüm ve kalıp yapısı

İkinci çözümde soyut bir liste sınıfı ve onda dolaşmaya yarayan soyut bir iterator sınıfı vardır (ekrandan, 00:20:00):

```
AbstractList  <------  Client  ------>  Iterator
CreateIterator()                       First()
Count()                                Next()
Append(Item)                           IsDone()
Remove(Item)                           CurrentItem()
...
   ^                                      ^
   |  (kalıtım)                           |  (kalıtım)
List      <--------------------------->  ListIterator
SkipList  <--------------------------->  SkipListIterator
```

Soyut listeye bir `CreateIterator()` metodu eklenir; bu bir sanal fabrika metodudur (virtual factory method). Bütün metotlar soyut olduğu için bu sınıf bir arayüz gibi de düşünülebilir. Önceki çözümdeki `Count`, `Append`, `Remove` gibi metotlar soyut olarak tanımlanır. Sonra soyut listenin somut alt sınıfları ve soyut iteratorun somut alt sınıfları oluşturulur. `List` için `ListIterator`, aradaki bazı elemanları atlayabilen `SkipList` için `SkipListIterator`, dairesel liste için dairesel liste iteratoru gibi kardeş sınıf çiftleri ortaya çıkar.

İstemci hangi veri yapısıyla uğraşırsa uğraşsın onu dolaşacak bir iterator elde edebilir. Iteratorun iç ayrıntılarını bilmeden, yalnız soyut Iterator'un arayüzünü kullanarak dolaşır. İstemci açısından her şey derli topludur. Ama belirli bir iterator sınıfı ile belirli bir liste arasındaki ilişki sürer ve sürmek zorundadır. Belli bir veri yapısında nasıl dolaşılacağının ayrıntıları en iyi o yapıya özel bir iteratorda sarmalanır.

Kalıp yapısı bu çözümün genelleştirmesidir. Bileşenlerden oluşan bir bütünü simgeleyen soyut bir yapı vardır; bu yapı üzerinde dolaşılmasını sağlayan `CreateIterator` metodunu sunar. Dolaşmayı sağlayacak sınıfın temel yetenekleri yine soyut bir üst sınıfta tanımlanır. Somut alt sınıflarda gereken kodlanır. İstemci soyut üst sınıflarla konuştuğu için yapı yeni alt sınıflarla istenildiği gibi genişletilebilir ve bu genişlemeler istemciyi etkilemez. Java'nın `java.util` paketindeki veri yapıları bu kalıbın biraz daha basitleştirilmiş hâlini kullanır.

2. Iterator Gerçekleme Kararları

Hoca, gerçekleme yaparken verilmesi gereken üç önemli karara dikkat çekti.

Birinci karar: iterator parçalar arasında dolaşırken parçaların durumunun değiştirilmesine izin verilecek mi? İzin verilmeyecekse iterator veri yapısının kendisi üzerinde değil, klonu üzerinde dolaşabilir. Böylece değişiklik yapılsa bile veri yapısındaki bileşenler etkilenmez.

İkinci karar: dolaşma algoritmasını kim gerçekleyecek? Örneğin ağaç yapısının LDR, DLR gibi farklı dolaşımları nerede kodlanacak ve nerede seçilecek: veri yapısında mı, iteratorda mı?

Üçüncü karar, ikincisiyle ilişkilidir: dolaşılacak nesneler istekçiye birer birer mi, yoksa hepsi birden mi verilecek? İstekçi her seferinde "next item" diyerek bir sonraki elemanı mı alacak, yoksa bileşik yapının elemanları bir dizi içinde bir araya getirilip istekçiye mi verilecek? Toplu verilirse istekçi o dizide istediği gibi serbestçe dolaşabilir; bu serbestliğin işe yarayıp yaramadığına bakılmalıdır.

Bu soruların "mutlaka şöyle olmalıdır" denilen bir yanıtı yoktur; problem ya da uygulama alanı neyi gerektiriyorsa öyle yapılır. Bütün elemanları birden vermek daha uzun sürer. Tek tek almak başarımı iyileştirebilir. Hele bileşenlerle istemci ayrı adres uzaylarındaysa, bileşenler bir ağ hattı üzerinden gidiyorsa, haberleşme masraflı ve yavaşsa ya da verilerin hazırlanması pahalıysa, tek tek alırken arka planda önceden çekme (prefetch) mekanizması kurmak yararlı olabilir.

> **Hocanın vurgusu:** Çok yabancı bir kalıp değildir, hepimizin bildiği bir şeydir; ama bu önemli ayrıntılarını bilmek gerekir.

3. Iterator Örneği: Birleşen İki Lokantanın Menüsü

3.1 Problem

Örnek Head First Design Patterns kitabından alınmıştır. Çevrim içi yemek siparişi işine girilmek isteniyor ve bu alanda hızlıca bir firma kurulacak. En hızlı yol satın almadır: biri kahvaltıda (Pancake House), biri yemekte (Diner) uzman iki firma satın alınır. Problemi kolaylaştırmak için menü elemanı sınıfı `MenuItem` iki firmada aynıdır; belki ikisi de açık kaynak bir çözümden yararlanmıştır. Ama menüyü dolaşma ve sunma açısından iki firma ayrı gerçekleme yapmıştır.

İki farklı menü gerçeklemesinden birleştirilmiş bir menü nasıl sunulur? Ortak bir kod yazılmazsa her yeni satın alınan firma için menü sunma koduna yeni bir seçenek eklemek gerekir; büyümek isteyen bir şirket bunu istemez. Ayrıca yeni gereksinimler işi karmaşıklaştırır. Vejetaryenler düşünülmüş ve hangi yemeklerin et içerdiği işaretlenmiştir. Kahvaltı menüsü, yemek menüsü ya da bütün menüyü göstermeye ek olarak "bütün seçenekleri göster" ya da "yalnız vejetaryen seçenekleri göster" isteği de çıkmıştır. Zamanla başka istekler de çıkabilir.

3.2 Ortak sınıf ve iki menü gerçeklemesi

`MenuItem` sınıfında bir menü elemanının adı (`name`), tanımı (`description`), vejetaryenlere uygunsa `true` olan `vegetarian` alanı ve fiyatı (`price`) vardır. Kurucu bunların hepsini alır; her biri için getter vardır. Elemanı tarif eden `toString()` şöyledir (ekrandan, 00:30:00):

```
public String toString() {
    String info = name + "\t" + description + "\t" + price;
    if( vegetarian )
        info += "\t" + "(Vegetarian)";
    return info;
}
```

Hoca derste "Vegetarian" yazımını düzeltip kontrol etti.

Diner'cılar biraz acemice davranıp liste değil dizi kullanmışlardır. Menülerinde şimdilik 6 seçenek olduğu için 6 elemanlık bir dizi oluştururlar. `addItem` metodu bir `MenuItem` oluşturup diziye ekler; dizi doluysa "ekleyemiyorum" der. Hoca, dizi dolunca daha büyük yeni bir dizi oluşturup eski elemanları ona kopyalamanın ve üye alanı yeni diziye bağlamanın da mümkün olduğunu, ama üzerinde durmayacağını söyledi. Menü elemanlarını elde etmek için `getMenuItems()` diziyi hiçbir işlemden geçirmeden olduğu gibi döndürür.

Pancake House'çular biraz daha akıllıca davranıp menüyü bir `ArrayList` üzerinde gerçeklemişlerdir. Kurucuda elemanlar oluşturulup listeye eklenir. `addItem` gelen bilgiden bir eleman oluşturup listeye ekler. `getMenuItems()` geriye `ArrayList` döndürür.

3.3 İlk çözüm: "quick and dirty"

İlk akla gelen çözüm, ortak bir kod olmadan iki ayrı gerçeklemeyi birleştirmektir. Hoca bu çözümü ders notu ve kod olarak paylaşmadığını, ama sakıncalarını görmek için hızlıca inceleyeceklerini söyledi. `WaitressQuickAndDirty` sınıfında iki menü türünden birer üye alan vardır ve kurucu ikisini de ilklendirir. Bütün menüyü sunarken önce "kahvaltı menüsünü sunuyorum" denir. Kahvaltı evinin `ArrayList`'i alınır, her elemanı tek tek alınıp yazdırılır. Sonra yemek evinden dizi alınır ve o da tek tek dolaşılıp yazdırılır. Yazdırma işi `printMenuItem` metodundadır; adı, fiyatı ve tanımı yazar.

Vejetaryen menü kodu öncekine çok benzer; yalnız yazdırmadan önce bir `if` denetimi vardır (ekrandan, 00:36:30):

```
ArrayList<MenuItem> breakfastItems = pancakeHouseMenu.getMenuItems();
for( int i=0; i<breakfastItems.size(); i++ ) {
    MenuItem item = breakfastItems.get(i);
    if( item.isVegetarian() )
        printMenuItem(item);
}
System.out.println("\nLUNCH");
MenuItem[] dinerItems = dinerMenu.getMenuItems();
for( int i=0; i<dinerItems.length; i++ ) {
    MenuItem item = dinerItems[i];
    if( item.isVegetarian() )
        printMenuItem(item);
}
```

Aynı sınıfta `printBreakfastMenu()` (`"MENU\n----\nBREAKFAST"` yazıp `ArrayList`'i dolaşır) ve `printDinerMenu()` metotları da vardır. Kahvaltı menüsünü yazdırmak, bütün menüyü göstermenin ilk yarısının aynısıdır; yemek menüsünü yazdırmak da ikinci yarısının aynısıdır.

Sakıncalar:

- İki ayrı yapı iki ayrı biçimde ele alınır.
- Normal menü ile vejetaryen menüyü yazdırma kodları birbirine çok benzer; tek fark bir `if` denetimidir. Bir hata yapılırsa ya da bir değişiklik istenirse iki metotta birden düzeltme gerekir.
- İleride kod kusurları (code smell) incelenirken buna yinelenen kod (duplicate code) denecek: hemen hemen aynı kod birden fazla yerde tekrarlanır.
- Acilen bir çözüm çıkarılmış olur, ama en ufak yenilikte iş zorlaşır.

En azından `printMenuItem` işi ayrı bir metottadır.

3.4 Iterator ile çözüm: tasarım

Farklı menülerde farklı biçimlerde dolaşıp kullanıcıya sunma ihtiyacı, Iterator kalıbını akla getirir. Diner ve Pancake House menülerinin iç yapısına müdahale etmeden bu karmaşıklıktan ön yüz soyutlanmalıdır. Tasarım şöyledir (ekrandan, 00:41:00):

```
«interface» Menu                  Waitress          «interface» Iterator<MenuItem>
+ createIterator() :     <------          ------>  + hasNext() : boolean
  Iterator<MenuItem>                                + next() : MenuItem
     ^ (gerçekler)                                      ^ (gerçekler)
DinerMenu          <------------------------>   DinerMenuIterator
PancakeHouseMenu   <------------------------>   PancakeHouseMenuIterator
```

Farklı menüler ortak bir arayüz altında toplanır; farklı menü dolaşımları da ortak bir arayüz altında toplanır. Üye alan ya da dolu (somut) metot eklemeye gerek olmayacağı için hoca `Menu`'yü arayüz olarak yazdı. Gerekirse soyut sınıfa çevrilebilir; öngörü yanlış çıksa bile düzeltmek iki kelimelik iştir. Pancake House ve Diner menüleri `Menu` arayüzünü gerçekler. Her birinde dolaşacak iterator sınıfları yazılır ve bunlar Iterator arayüzünü gerçekler.

3.5 Iterator ile çözüm: kod

`Menu` arayüzü yalnız bir iterator oluşturma metodu ekler (ekrandan, 00:40:00):

```
package dp.iterator.example;

public interface Menu {
    public java.util.Iterator<MenuItem> createIterator();
}
```

Iterator, Java'nın generik sınıf desteğini kullanacak biçimde kodlanmıştır. Java'nın kendi Iterator'u en azından "sonraki elemanı getir" metoduna sahiptir ve bu iş için yeterlidir. Ayrıca `java.util.Iterator` altyapısı Java döngüleriyle birlikte kullanılabilir. Böylece hem Java'nın kolaylıkları hem de Iterator kalıbının önerdiği altyapı kullanılmış olur. Küçüktür-büyüktür işaretleri arasındaki `MenuItem`, bu iteratorun `MenuItem` türünden elemanlar getireceğini gösteren generik gösterimdir.

`DinerMenu` ve `PancakeHouseMenu` sınıflarına `implements Menu` eklenir ve `createIterator()` kodlanır. `PancakeHouseMenu.createIterator()` bir `PancakeHouseMenuIterator` oluşturup döndürür. Iterator, dolaşılacak varlıkları istemciye iletebilmek için bilmek zorundadır; bu yüzden menü elemanları ona parametre olarak verilir. `DinerMenu` de kendi dizisini parametre vererek bir `DinerMenuIterator` oluşturur. Elemanlar bir menüde `ArrayList`'te, ötekinde dizide tutulsun; hangi iteratorun hangi yapıyla çalıştığını iteratorlar bilir.

Hoca, burada Java'nın Iterator arayüzünün sunduğu özelliklerin yettiğini, ekstra dolaşım seçeneği gerekmediğini söyledi. Kullanılan dil böyle bir arayüz sunmasaydı onu da kendimiz kodlamak zorunda kalırdık. Java'nın Iterator'u `hasNext()` ve `next()` metotlarına sahip olmayı gerektirir; ek olarak `remove()` da vardır. Bu örnekte `remove` kullanılmadı, ama menüden eleman silmek gerekirse kullanılabilir.

`DinerMenuIterator` dizi ile çalıştığı için `MenuItem` dizisini üye olarak tutar; kurucu onu ilklendirir. Hangi elemanda olunduğunu tutan bir `position` sayacı vardır (ekrandan, 00:44:20):

```
public DinerMenuIterator(MenuItem[] list) {
    this.list = list;
}

public MenuItem next() {
    MenuItem menuItem = list[position];
    position = position + 1;
    return menuItem;
}

public boolean hasNext() {
    if (position >= list.length || list[position] == null) {
        return false;
    } else {
        return true;
    }
}
```

`next()` o anki elemanı alır, sayacı bir artırır ve elemanı döndürür. `hasNext()` listenin sonuna ulaşılıp ulaşılmadığına bakar.

> **Ders sonrası not:** Hoca `hasNext()` için "sona ulaşmamışsa false, ulaşmışsa true döndürüyor" dedi (00:44:28; `PancakeHouseMenuIterator` için 01:45:42 civarında da aynı biçimde). Ekrandaki kodda doğrusu tersidir: konum dizinin boyunu geçmişse ya da o konumdaki eleman `null` ise (yani sona ulaşılmışsa) `false`, aksi hâlde `true` döner.

Java'nın iterator yapısının bir kuralı vardır: bir kez `next()` denmeden eleman silmeye izin vermez. Örnek kod bu kurala uyacak biçimde yazılmıştır.

`PancakeHouseMenuIterator` da `Iterator<MenuItem>`'ı gerçekler; bu kez elemanlar bir `ArrayList` içindedir. Konum yine sıfırdan başlar ve kurucu listeyi alıp ilklendirir. `next()` `ArrayList`'in

belli bir konumdaki elemanı veren `get` metodunu kullanır, konumu bir artırır ve elemanı döndürür. `hasNext()` bu kez listenin `size()` metoduyla konumu karşılaştırır.

Böylece dizi üzerinde dolaşmanın karmaşıklığı `DinerMenuIterator`'a, `ArrayList` üzerinde dolaşmanın karmaşıklığı `PancakeHouseMenuIterator`'a alınmış olur. İkisinin sunduğu ana hizmet aynıdır: Java'nın iteratoruyla uyumlu olarak `hasNext` ve `next` ile menü elemanları arasında dolaşmak.

3.6 Yeni Waitress sınıfı

Yeni `Waitress` sınıfı iki ayrı menüyle uğraştığını hâlâ bilir, ama onları eş biçimli olarak okuyup müşteriye sunabilir. `printMenu()` önce "kahvaltı menüsünü basıyorum" deyip kahvaltı menüsünün iteratorunu alır ve `printMenu(Iterator)`'a verir; sonra yemek menüsünün iteratoruyla aynı metodu çağırır. Kod şöyledir (ekrandan, 00:50:00):

```
private void printMenu(Iterator<MenuItem> iterator) {
    while (iterator.hasNext()) {
        MenuItem menuItem = iterator.next();
        printMenuItem(menuItem);
    }
}

private void printMenuItem( MenuItem menuItem ) {
    System.out.print(menuItem.getName() + ", ");
    System.out.print(menuItem.getPrice() + " -- ");
    System.out.println(menuItem.getDescription());
}

public void printVegetarianMenu() {
    System.out.println("\nVEGETARIAN MENU\n----\nBREAKFAST");
    printVegetarianMenu(pancakeHouseMenu.createIterator());
    System.out.println("\nLUNCH");
    printVegetarianMenu(dinerMenu.createIterator());
}
```

`printVegetarianMenu(Iterator<MenuItem> iterator)` de benzer biçimde iteratorda dolaşır ve eleman vejetaryense `printMenuItem` ile yazdırır.

`printMenuItem`'daki üç satırlık komut `printMenu` içine de alınabilirdi. Hoca bunu ayrı tutmanın sakıncası olmadığını, hatta genelde tavsiye edilenin bu olduğunu söyledi. `printMenu` bütün menü elemanlarını basma işini, `printMenuItem` ise bir elemanın nasıl basılacağının ayrıntısını gerçekler.

> **Hocanın vurgusu:** En çekirdek, en odaklanmış işleri ayrı metotlarda sarmalamanın hiçbir sakıncası yoktur, hatta yararı vardır. Bunun yararı `printVegetarianMenu`'de görülür: menü elemanı basma kodları tekrar yazılmadan hazır metot çağrılır.

3.7 Kalan sakınca ve olası genişlemeler

Sonuç epey derli topludur. Hocanın gördüğü tek sakınca şudur: `printMenu` ile `printVegetarianMenu` hâlâ birbirine epey benzer. Tek fark, iteratordan `next` alındıktan sonra gelen ekstra bir `if` komutudur. Bu sorunu çözmek için iterator oluşturulurken ya da kullanılırken ona "bütün üyelerde dolaş" ya da "vejetaryen üyelerde dolaş" diye bir yapılandırma verilebilir.

Ama hazır `java.util.Iterator` kullanıldığı için ona böyle bir ekstra metot eklenemez. Java'nın iteratoru yerine ondan yola çıkarak kendi iterator sınıfımızı yazabiliriz; bunun için biraz daha şey öğrenmek gerekir. Sırf vejetaryen menü için bu yatırıma değmez; vejetaryen olup olmama bir Boolean ile çözülebilir. Ama zamanla glutensiz yemekler, çocuk menüsü, çölyak hastaları için menü gibi seçenekler gelecekse `setVegetarian(true)` gibi bir metot yetmez. Bunu bir enum ile yapmaya kalkılırsa switch-case yapısına gidilir. O durumda oturup bu varyasyonları daha iyi modellemek, ya iyi bir nesneye dayalı tasarım yapmak ya da öğrenilen ve öğrenilecek kalıplardan birini daha uygulamak gerekir. Hoca bu dalı bu derste açmadı.

4. Memento Kalıbı

4.1 Ad ve amaç

Memento, sözlük anlamıyla bir anı parçasıdır. Hoca kalıbı aynı adlı bir filmle tanıttı. Filmin Türkiye'de kullanılan afişini gösterdi; filmin Türkçe adı "Akıl Defteri"dir. Filmin kahramanı uzak geçmişini hatırlar ama yakın geçmişi, birkaç gün öncesini aklında tutamaz. Bu yüzden sağına soluna küçük notlar alır ve onlara göre hareket eder. Hoca filmin ayrıntısını anlatmadı.

Kalıbın temel fikri, bir varlığın anlık görüntülerini (snapshot) alıp bunlar arasında işlem yapabilmektir, ama biraz daha ayrıntısı vardır:

- Bir nesnenin durum bilgisini elde etmek istiyoruz. Böylece nesnenin değişik zamanlardaki durumlarını saklayıp gerekirse önceki hâllerinden birine dönebiliriz.
- Bunu yaparken sarmalama (encapsulation) ilkesini bozmak istemiyoruz.
- Alınan anlık bilgilere dışarıdan müdahale edilememesini de istiyoruz. Bir nesnenin belli bir andaki durum bilgisi değiştirilememeli, yalnız bütün olarak alınıp başka bir anda aynı türden bir varlığa verilebilmelidir. Dışarıdan sahte (forge edilmiş), değiştirilmiş durum bilgisi verilememelidir.

Durum bilgileri toplu alınınca başka işler de yapılabilir: nesneler diske saklanabilir, ağ bağlantısı üzerinden aktarılabilir, geçirdiği aşamaların güncesi tutulabilir. Bir işlem (transaction) mantığında her ara adımda değişiklikler olur; bir sorun çıkınca hiçbir şey olmamış gibi işlemin başına dönmek (rollback) için Memento faydalı bir araçtır. Bir nesne üzerinde düzenleme yaparken geri alma (undo) ya da yedekten geri yükleme için de yararlıdır.

Hoca bir parantez daha açtı: belki durum bilgisinin hepsine değil, yalnız bir kısmına ihtiyaç vardır. İster hepsi ister bir kısmı olsun, kalıp bunu ayrıntıları açığa çıkarmadan ve mümkünse dışarıdan değiştirmeyi engelleyerek yapmaya yöneliktir.

4.2 Kalıp yapısı ve bileşenler

Kalıp yapısı şöyledir (ekrandan, 01:20:00):

```
Originator                      Memento              memento
----------------------  - - ->  -----------  <------------<> Caretaker
SetMemento(Memento m)           GetState()
CreateMemento()                 SetState()
----------------------          -----------
state                           state

CreateMemento notu: return new Memento(state)
SetMemento notu:    state = m->GetState()
```

Originator, durum bilgisiyle ilgilendiğimiz varlıktır. Bir `state` alanı ve çeşitli metotları vardır; kalıpla ilgili olanlar şunlardır. `CreateMemento()` varlığın o andaki durum bilgisinin bir resmini çeker ve bir Memento olarak döndürür. `SetMemento(m)` parametre olarak bir Memento alır ve durumu o Memento'nun söylediği hâle getirir. Durum bilgisinin tamamı ya da ilgilenilen kısmı Memento adlı sınıfa alınır. Hoca burada sözcüğü düzeltti: durum taşınmaz, kopyalanır.

Memento'nun durum için `GetState` ve `SetState` metotları vardır. Ama bunları kullanması gereken yalnız Originator'dır, başkası değil. Caretaker bu Memento'ların bir koleksiyonunu tutar. Şemadaki elmas (aggregation) işaretinden bunun bire-çok ilişki olduğu anlaşılmalıdır. Caretaker belki güncelerini tutar, belki geri alma işlemleri yapar, ama durumu değiştirememelidir.

Slayttaki bileşen tanımları (ekrandan, 01:20:00):

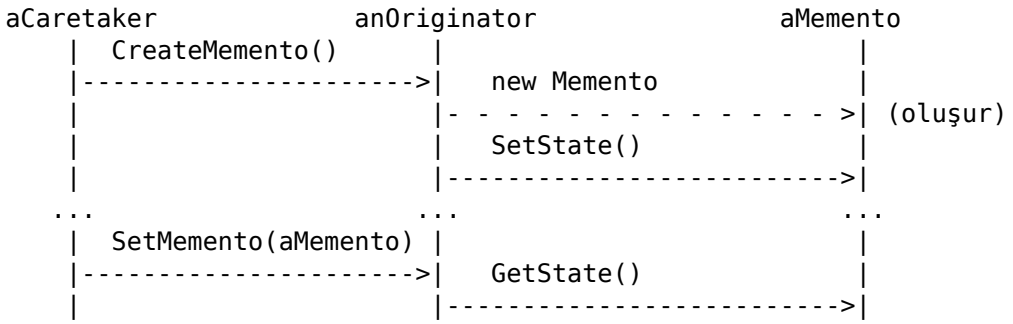
- Originator: durum bilgisi saklanacak olan nesne.
- Memento: durum bilgisinin gerekli kısımlarını saklar.
- Caretaker: gerekli anlarda durum bilgisinin bir kopyasını alır ve elindeki kopyalardan birini geçerli durum hâline getirir.

Caretaker geçerli durumu Memento'yu doğrudan değiştirerek değiştirmemelidir. Elindeki Memento'lardan birini Originator'a verir ve bunu onun `SetMemento` metodu üzerinden yapar.

Hoca, söylemesi kolay olsa da bunu kodlamanın kolay olmadığını belirtti. Bir metot public ise herkes, private ise başka kimse çağıramaz. “Bu metot şu sınıflar tarafından çağrılabilsin, şunlar tarafından çağrılamasın” diyen seçici bir mekanizma bildiği nesneye dayalı dillerde yoktur. Smalltalk’ı o kadar incelemediği için onu bilemediğini söyledi. Ama iç sınıf mantığıyla bazı şeyler yapılabilir.

4.3 Sıralama şeması

Slayttaki sıralama şeması (sequence diagram) şöyledir (ekrandan, 01:22:00):



Caretaker, Originator’dan bir Memento ister. Originator bir Memento nesnesi oluşturur ve `SetState` ile durumu olması gerektiği gibi ayarlar. Hoca, şemada dönüş oklarının çizilmediğini ekranda kendisi tamamladı: `new Memento` sonucu Originator’a döner, Originator `SetState` ile “durum bilgim budur, al bunu, kendi içinde sakla” der, metot bitince Memento Caretaker’a iletilir. Ayrıca kurucu çağrısı (`new`) sıralama şemalarında normalde kesikli çizgiyle çizilmez. Hoca bu alternatif gösterimleri, Dörtlü Çete’nin kitabının UML gösterimleri henüz tam oturmamışken yazılmış olmasına bağladı.

Bir süre sonra Caretaker “artık bu hâle geri dönülmesini istiyorum” der ve daha önce aldığı Memento’yu parametre vererek Originator’a `SetMemento` çağrısı yapar. Originator Memento’ya durum bilgisini sorar (`GetState`), aldığı bilgiyle kendi içindeki setter metotlarını çağırarak durumunu günceller ve metotlar biter.

4.4 Ne zaman kullanılır?

Kalıp, nesnenin durum bilgisinin tümünün ya da bir kısmının daha sonra o duruma geri dönülebilmek için saklanması gerektiğinde kullanılır. Ama bu iş başka yollarla da yapılabilir: durumu oluşturan üye alanlar normalde bir yapıda toplanabilir. Duruma ya da bir kısmına alınacak doğrudan bir işaretçi, ya da public metotlarla erişim veri gizliliğini bozar. Bunun istenmediği durumlarda Memento kalıbını uygulamaya zahmet edilir; aksi hâlde ek kalıp kurmaya kesinlikle gerek yoktur.

> **Hocanın vurgusu:** Memento’nun amacı veri gizliliği ilkesini bozmamaktır. Bunun için Memento’nun ilgili metotlarına yalnız Originator’dan erişilebilmelidir.

5. Memento’da Veri Gizliliğinin Gerçeklenmesi

5.1 Dillere göre seçenekler

C++’ta `friend` adlı bir yapı vardır; her dilde yoktur. Memento’nun metotları private yapılır ve Originator, Memento’nun arkadaşı (friend) ilan edilir. Arkadaş sınıf, arkadaşı olduğu sınıfın private metotlarına erişebilir.

> **Ders sonrası not:** Hoca sözdizimini “public class memento, friend originator” diye kabaca söyledi. C++’ta bu, Memento sınıfının içinde `friend class Originator;` bildirimiyle yapılır.

Java’da birinci seçenek paket (package) kuralıdır. Bir üye alana ya da metoda public, private ya da protected yazılmazsa otomatik olarak paket erişimi devreye girer. Bu üyeler aynı paketteki sınıflara açık, başka paketteki sınıflara ise private gibidir. Ama kaynak kod class dosyası olarak dağıtılırken dikkat gerekir. Jar dosyası yapılırken içindeki paketler `sealed` olarak işaretlenmelidir; bu, jar aracına bir parametre olarak verilir. Paket ya da jar mühürlenmezse dışarıdan

biri istediği bir sınıfı oradaki paketlerden birine enjekte edebilir ve kimse buna hayır demez. Böyle eklenen bir sınıf paket erişimli bütün metotlara erişebilir hâle gelir ve Memento'nun veri gizliliği önlemi delinir.

Daha sağlam bir yol, Java'da iç sınıf kullanmaktır: Memento, Originator'ın iç sınıfı yapılır. Hoca önce private iç sınıf demeyi düşündü, sonra public de olabileceğini söyledi. Bir püf noktası daha vardır: durum (`State`) kısmı Java'da mutlaka `Cloneable` olarak işaretlenmeli ve kopyası döndürülmelidir. Hoca, bu iç sınıflı çözümü ilk yaptığında testlerin bir türlü geçmediğini, uzun süre araştırdığını ve sonunda klonlamanın gerektiğini keşfettiğini anlattı. Paket erişimli çözümü ve C++ çözümünü göstermedi; yalnız iç sınıf çözümünü kod üzerinde gösterdi.

5.2 State sınıfı

Kod `dp.mementoV2.innerClass` paketindedir. Durumu basitçe tek bir string olarak tuttu; daha fazla üye alan konabilir, ama örnek için gerek yoktur. Kalıp şemasında durum tek bir üye alandır, ama hoca gerçeğe daha yakın bir altyapı için ayrı bir `State` sınıfı yazdı (ekrandan, 01:30:00):

```
package dp.mementoV2.innerClass;

public class State implements Cloneable {
    private String durum;

    public String getDurum() {
        return durum;
    }

    void setDurum(String durum) {
        this.durum = durum;
    }

    public State clone() {
        State s = new State();
        // ... (kopyanın durumu bu nesnenin durumuna eşitlenir, kopya döndürülür)
    }
}
```

Getter public'tir; setter ise paket erişimlidir. Hoca setter'ı paket erişimli yapmanın tek sorun noktası olduğunu, ama Originator'da iç sınıf kullanılacağı için dışarıdan basitçe set yapılamayacağını söyledi. `clone()` metodu yeni bir `State` nesnesi oluşturur, onun durumunu kendi durumuna eşitler (kopyalar) ve kopyayı döndürür. Hoca derste bir an `getDurum()`'un mu, kopyanın mı döndürüleceğini karıştırdı ve klonun döndürülmesi gerektiğini teyit etti.

5.3 Originator ve iç sınıf Memento

Originator'ın `State` türünden bir `state` alanı vardır; kurucusu boş bir `State` ile ilklendirir. Kalıbın gerektirdiği metotlar `createMemento()` ve `setMemento()`'dur:

- `createMemento()` önce boş bir Memento oluşturup ilklendirir. Sonra onun durumunu, kendi durumunun kopyasını oluşturarak ayarlar ve Memento'yu döndürür.
- `setMemento(m)` aldığı Memento'nun durumunu alıp kendi `state` alanına atar.

Hoca, geri dönmenin doğru çalıştığını gösterebilmek için bir de `modify` metodu ekledi. Bu bir tür `setState` metodudur. Kodun görünen kısmı şöyledir (ekrandan, 01:32:50):

```
public void modify( String str ) {
    state.setDurum(str);
}

public String toString() {
    return state.getDurum();
}
```

```
public class Memento {
    private State state;

    private State getState() {
        return state;
    }
    // ...
}
```

`toString()` nesnenin durumunu göstermek için durumdaki metni döndürür. `Memento` sınıfı, `Originator` sınıfı bitmeden onun bloğu içinde açılmıştır. Hoca, private demeye gerek olmadığını, public'in yettiğini söyledi. Dış sınıftan Memento nesnesine erişim gerektiği için public daha esnektir ve bir soruna yol açmaz. Memento'da `state` alanı ve onun için başka bir önlem almaya gerek kalmadan erişim metotları vardır.

İç sınıflar kullanılırken, iç sınıfın bulunduğu sınıfla onu kullanan sınıf aynı paketteyse bile iç sınıf `import paket.DisSinif.IcSinif` biçiminde import edilmelidir. Aksi hâlde "bu sınıfı bulamadım" derleme hatası alınır. Hoca Memento'yu Caretaker'da doğrudan kullanabilmek için public yaptığını söyledi.

5.4 Caretaker ile deneme

`Caretaker.test()` metodu şunları yapar:

1. `Originator` türünden bir `subject` oluşturulur ve `modify("ilk durum")` ile ilk durum verilir.
2. `subject`'ten bir Memento (`mem`) istenir; bu Memento ilk durumu saklar.
3. `subject`'in durumu `modify("yeni durum")` ile değiştirilir.
4. İlk duruma dönmek için `subject.setMemento(mem)` çağrılır.
5. `subject` yazdırılır.

`main` bir Caretaker nesnesi oluşturup test metodunu çağırır. Çıktı "ilk durum"dur. `setMemento` çağrılmasaydı "yeni durum" yazılırdı; Memento verildiği için eski, yani ilk durum yazılır. Olması gereken de budur.

5.5 Klonlama gerekliliğinin gösterilmesi

Hoca sonra `State`'in klonlanması gerektiğini gösterdi. `clone()` metodunu kopya oluşturmak yerine `return this;` yapacak biçimde değiştirdi; yani nesne kendisini döndürdü. Caretaker tekrar çalıştırıldığında `setMemento` yapılmasına rağmen çıktı "yeni durum" oldu: aktarım gerçekleşmedi. Hoca kopya döndüren gövdeyi geri koyunca çıktı tekrar "ilk durum" oldu. Hoca, arka plandaki mekanizmanın neden klonsuz çalışmadığını tam anlayamadığını, ama çalışan çözümü bulduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Klonlamanın nedeni büyük olasılıkla takma ad (aliasing) sorunudur. `clone()` `this` döndürürse Memento ile Originator aynı `State` nesnesini paylaşır. Ardından gelen `modify("yeni durum")` bu ortak nesneyi değiştirdiği için Memento'daki "eski" durum da değişmiş olur. Kopya verildiğinde Memento bağımsız bir `State` tutar. Bu yüzden asıl gereken, `Cloneable` işaretinden çok, Memento'ya ayrı bir kopyanın verilmesidir.

5.6 Memento'nun bütünlüğü ve kırma denemesi

Memento'nun bütünlüğü garanti altında mı? Hoca Caretaker'da elindeki Memento'nun bir metodunu çağırmaya çalıştı. Eclipse'in önerdiği metotların hepsi Java'nın kozmik üst sınıfı `Object`'in metotlarıydı. Memento sınıfındaki hiçbir ek metoda Caretaker'dan erişilemiyordu. Caretaker Memento'nun işaretini alabilir, ama onun hiçbir metodunu çağıramaz; bu iç sınıf olması sayesindedir. Böylece Memento üzerinden sahte durum verme gibi bir doğrudan müdahale yolu kalmaz. Originator'dan alınan bir Memento ona geri verilebilir, ama Caretaker'ın kendi oluşturduğu sahte bir Memento verilemez.

Sonra kırma denemesi olan `testBroken()` metoduna geçildi (ekrandan, 01:40:00):

```
public void testBroken( ) {
    Originator subject = new Originator( );
    subject.modify("ilk durum");
    subject.modify("yeni durum");
    //bir süre sonra...
    Originator fakeSubject = new Originator( );
    ... "sahte durum");                 // satır 29, başı örtülü
    ...akeSubject.createMemento();      // satır 30, başı örtülü
    ... memFake );                      // satır 31, başı örtülü
    System.out.println(subject);
}
```

Ekranda 29-31. satırların başı Eclipse'in açılır penceresiyle örtülüdür. Hocanın anlatımına göre bu satırlarda `fakeSubject`'e "sahte durum" verilir, ondan `memFake` adlı bir Memento alınır ve bu Memento `subject.setMemento(...)` ile asıl nesneye verilir. İkinci bir Originator (`fakeSubject`) oluşturulur ve ona "sahte durum" verilir. Ondan bir Memento istenir ve bu Memento asıl `subject`'e `setMemento` ile verilir; sonra `subject` yazdırılır.

Çalıştırıldığında kırma denemesi "sahte durum" yazdırarak başarılı oldu. Yani Memento doğrudan değiştirilemez, ama bu yöntemle dolaylı olarak değiştirilmiş olur. Başka bir Originator oluşturup durumunu istenen hâle çekmek, ondan bir Memento alıp önceki nesneye vermek şu an hiçbir şey tarafından engellenmez.

5.7 Önerilen ek önlem

Hoca biraz daha düşünülürse bunun da engellenebileceğini söyledi ve bir yol önerdi:

- Originator'a bir kimlik (ID) üye alanı konur. Bu alan private ve final'dır ve kurucuda rastgele bir sayıyla ilklendirilir. Getter metodu verilmez; dışarıdan hiçbir biçimde okunamaz.
- İç sınıf Memento'ya kurucuda bu rastgele seri numarası verilir. Alanın private olması sorun değildir: bir sınıfın metodu o sınıfın private ya da public bütün üyelerine erişebildiği gibi, iç sınıf da dış sınıfın private üyelerine erişebilir.
- Memento'ya bu numara için bir getter konur.
- `setMemento` önce gelen Memento'nun söylediği numaraya bakar. Numara Originator'ın kendi numarasıyla aynıysa durumu atar, değilse atamaz.

Hoca bu mantığı kodda "ID vermeyen mementoyu set için kabul etme" diye bir açıklama olarak not etti, ama kodlamadı. Gerekmiyorsa iç sınıf çözümünün de yeterli sayılabileceğini ekledi.

6. Memento Örneği: Geri Alma ve Yineleme

Çözülecek problem: bir belge işleme programında geri alma (undo) ve yineleme (redo) işlemleri. Slayttaki tasarım şöyledir (ekrandan, 01:46:30):

```
Controller  1 ------> *  Document                          DocumentMemento
                         - currentState : DocumentMemento - documentState
                         - states : List<DocumentMemento>      ^ *
                         - position : int                      |
                         + undo()                    1 -------+
                         + redo()
                         + anyOperation()

Not (Controller): DocumentMemento ile hiçbir şekilde ilişkisi yok.
```

Not kutusundaki kod:

```
public void undo( ) {
    position -= 1;
    currentState = states.get(position);
}
public void redo( ) {
    position += 1;
    currentState = states.get(position);
```

```
}
public void anyOperation( ) {
    states.add(currentState);
}
```

Uygulama (Controller) belgelerle uğraşır. Belgenin mevcut (en son) durumu da geçmiş durumları da belgenin kendisinde saklanır. Memento'ların yönetimini Controller da yapabilirdi, Document da; burada Document tercih edilmiştir. O anki konumu tutmak için `position` alanı vardır. `undo()` bir adım geri gitmek için konumu bir geri çeker ve `states` listesinden o konumdaki durumu getirir. `redo()` konumu bir ileri alıp sıradaki durumu getirir. Belgeyi değiştiren her işlemin (`anyOperation`) son komutu olarak durumun `states` listesine eklenmesi unutulmamalıdır.

Hoca bu örneğin kodunu yazmadı. Memento'nun kodlanmasını önceki örnekte yeterince incelediklerini, artısını eksisini konuştuklarını söyledi. Bu örnekte bu metotların dışında yapılacak çok şey olmadığını belirtti.

7. Observer Kalıbı

7.1 Amaç ve örnek

Aradan sonra Observer kalıbına geçildi. Amaç, bir nesnenin durumunda bir değişiklik olduğunda o nesneyle ilişkili bütün diğer nesnelerin bu değişiklikten haberdar edilmesidir. Böylece diğer nesneler neyin değiştiğini anlayıp gerekli eylemleri alabilir. Haberdar etme olayları ve ilişkilerin kurulup bozulması kodlama anında değil, çalışma anında değiştirilebilmelidir. İstenildiği kadar tekrarlanabilmeli, mevcut bir ilişki bozulup yerine yenisi kurulabilmelidir. Hangi gözleyicilerin ilişkide olacağı zaman içinde değişebilir.

Örnekte bir hesap tablosunda bir veri vardır; hoca burada MVC'ye girmeyeceklerini, yalnız Observer yönünden bakacaklarını söyledi. Asıl ilgilenilen varlık (subject) bu veridir. Onu izleyen başka varlıklar vardır: bu veriden hazırlanmış bir pasta grafik (pie chart) ve bir çubuk grafik (bar chart). Tablodaki rakamlar değişince grafikler de güncellenmelidir. Slaytta koyu gri yapılmış 50-30-20 değerleri Y satırındaki subject'e aittir; X ve Z satırları da benzer biçimde izlenebilir. Bir değişiklik olduğunda onunla ilgilenen bütün gözleyicilerin (observer) güncellenmesi sağlanmalıdır.

7.2 Kalıp yapısı

Önerilen yapı şöyledir (ekrandan, 02:10:00):

```
Subject (soyut)          observers          Observer (soyut)
Attach(Observer)    ------------------->    Update()
Detach(Observer)                              ^
Notify()                                      |
   ^     Notify notu: for all o in observers { o->Update() }
   |                                          |
ConcreteSubject     <--------------------   ConcreteObserver
GetState()                subject           Update()
SetState()                                  observerState
subjectState

GetState notu: return subjectState
Update notu:   observerState = subject->GetState()
```

Durumuyla ilgilenilen subject'ler ve onların durumlarını gözleyen, durumlarındaki değişikliklerle ilgilenen observer'lar vardır. Bir subject birden fazla observer tarafından gözlenebilir; bir observer birden fazla subject'i gözleyebilir; arada çoka-çok ilişki vardır. Bütün bunların farklı subject ve observer türleri arasında kullanılabilmesi için ortak bir altyapı gerekir: soyut Subject üst sınıfı ve soyut Observer üst sınıfı.

Subject ve observer türlerinden bağımsız olarak şu ortak işler yapılır: bir varlığa gözleyici iliştirmek (`Attach`), bir varlıktan gözleyiciyi çıkarmak (`Detach`) ve durum değişince gözleyicileri haberdar etmek (`Notify`). Bu altyapı Subject sınıfında kurulur. Observer sınıfında ise basitçe

bir `Update` metodu vardır. Observer bir arayüz olabilir; burada somut metot kodlamak gerekmez. Ama soyut Subject sınıfında `Attach`, `Detach` ve `Notify` somut metot olmak zorundadır. Farklı subject türleri Subject'ten, farklı gözleyiciler Observer'dan kalıtımla üretilir.

`Notify`'ın gerçeklemesi basittir. Observer nesneleri ister listede ister dizide tutulsun, önceki Iterator örneğindeki gibi iterator alınır, hepsinde dolaşılır ve her birinin `Update` metodu çağrılır. Observer'ın bilgiyi subject'ten nasıl alacağı ise pull ve push modellerine bağlıdır.

Somut subject'lerin durumu gözleyicilere iletmek için çeşitli getter ve setter metotları olur. Bütün durumu birden aktarmak yerine, farklı gözleyiciler farklı alanlardaki değişikliklerle ilgilenebilir; bu yüzden ilgili getter'lar kullanılır. Bir observer belli bir subject'in belli bir bilgisiyle, başka bir observer başka bilgileriyle ilgilenebilir. Bu yüzden somut bir observer belli bir türden somut subject ile ilişkide olmak zorundadır; burada bir sınıf bağımlılığı vardır. Bütün somut subject'lerde ortak bir metot varsa, o metot soyut Subject'e somut ya da soyut metot olarak konabilir. O zaman somut observer, somut subject yerine soyut Subject türü üzerinden de iletişim kurabilir.

Hoca şemadaki `subject` ilişkisinin isteğe bağlı olabileceğini söyledi. Bir observer, somut subject'e "durumun ne oldu" diye sormak zorunda olmayabilir. Durum, Memento'daki gibi ayrı bir sınıfta sarmalanmışsa observer somut subject ile değil, o durum nesnesiyle ilişkili olabilir.

Hoca şemadaki `observerState = subject->GetState()` notunu biraz yanıltıcı (misleading) buldu. Observer'ın durumu subject'in durumuyla aynı ya da ona bağımlı olmak zorunda değildir. Burada subject'in durumu doğrudan alınıp observer'ın durumuna atanmıştır; bu daha sınırlı bir senaryodur. Somut observer, subject'te neyin değiştiğini alıp onunla çeşitli işlemler yapar; kendi alanlarına doğrudan atamak zorunda değildir. Aksi hâlde observer biraz Proxy gibi olur.

8. Observer Gerçekleme Ayrıntıları

8.1 Pull ve push modelleri

Durum bilgisinin gözleyicilere gönderilmesi için iki modelden biri kullanılır. Birinin avantajı ötekinin dezavantajıdır. Slayttaki maddeler şöyledir (ekrandan, 02:20:00):

- Pull model:
  - (+) Gözlenen yalnız gözleyicileri haberdar eder; gözleyiciler ilgili durum bilgisini gözlenenden alır.
  - (+) Gözlenenin gözleyicilerle bağlantısını azaltan bir çalışma biçimidir.
  - (+) Gözleyici durum bilgisinin yalnız istediği kısmını almayı seçebilir.
  - (–) Gözleyici gözlenende nelerin değiştiğini kendisi çıkarmak zorunda kalır.
- Push model: gözlenen, gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir.

Pull modelde subject yalnız "ben değiştim" der. Gözleyiciler subject'in çeşitli metotlarını çağırarak değişen durumu elde eder. Bağlantının azalmasının nedeni şudur: subject yalnız soyut Subject'in `Notify` metodunu çağırır, bu da soyut Observer üzerinden bütün gözleyicilere gider. Subject'in hangi somut observer ile konuştuğunu bilmesi gerekmez. Somut observer yine somut subject ile iletişime girer; yani bağlaşım hafifler ama yok olmaz. En azından subject'ten observer'a giden ilişki `Update` çağırmakla sınırlı kalır.

> **Hocanın vurgusu:** İlişki var mı? Var. İster bir metodunu çağırsın ister on metodunu, ister bir nesneyle ister on nesneyle ilişkili olsun, fark etmez; yine bir bağlaşım sayılır. Çeşitli alternatif bağlaşım ölçütleri tanımlanabilir, ama "ilişki var mı, var" tanımı değişmez.

Yine de observer'ın diğer metotlarında yapılacak değişikliklerin subject'i etkileme olasılığı düşer. Üçüncü avantaj olarak, farklı observer türleri farklı subject'lerin farklı durum bileşenleriyle ilgilenebilir; her biri yalnız ilgilendiği alanları alır. Subject ve observer'lar farklı yerlerde, bir haberleşme ağıyla birbirine bağlı süreçler ya da nesnelerse daha az paket gider, daha az masraf olur ve haberleşme daha çabuk tamamlanır.

Dezavantajı, gözleyicinin neyin değiştiğini kendisinin keşfetmek zorunda olmasıdır; subject yalnız "değiştim" der. Örneğin bir gözleyici üç alanla ilgileniyor olsun. Gözleyicinin ilgilenmediği bir alan değişse bile bildirim bütün gözleyicilere gittiği için ona da gider. Gözleyici

subject'i sorgular, ilgilendiği üç alanın da değişmediğini görür ve belki bir iş yapmaz. Ya da "değişmiş olsun olmasın bu işi yapıyorum" diyebilir; o zaman yapılan işin çok uzun ya da masraflı olmaması gerekir ki boşuna tekrar zarar getirmesin.

Push modelde durum tam tersidir. Subject, gözleyicilere durum değişikliği hakkında ayrıntılı bilgi gönderir. En basit biçimde subject bütün durumunu iletir: `Notify` ile birlikte parametre olarak subject'in kendisi ya da bütün durumu gider. Dezavantajı, haberleşme masrafının daha fazla olmasıdır: gözlenen nesnenin bütün durumu gittiği için daha çok bayt gider, haberleşme daha pahalı ve uzun olur. Durumla birlikte bir değişiklik maskesi (change mask) de gönderilebilir. Örneğin nesnenin 10 alanı varsa 10 bitlik bir alan gönderilir. Her gözleyici ilgilendiği konumdaki bitin 0'dan 1'e döndüğünü görürse neyin değiştiğini kendisi incelemek zorunda kalmaz. İlgilendiği alan değişmişse alır, değişmemişse hiç almaz; kod daha rahat yazılır.

Avantajlar ve dezavantajlar birbirinin tam tersidir; problem neyi gerektiriyorsa o seçilir.

8.2 Basitleştirmeler ve tutarlılık

Belli bir türden subject'lerin hepsinin yalnız belli türden observer'ları olacaksa kalıpta bazı basitleştirmeler yapılabilir. Slayttaki bir madde de bunu söyler: aynı türden gözlenen nesnelerin tümünün gözleyicileri ortak olsun istenirse, gözleyiciler statik bir veri yapısında tutulabilir (ekrandan, 02:20:00). Ama bunun hep böyle kalacağının garantisi yoktur; farklı türler de gelebilir.

> **Hocanın vurgusu:** Bu çok hafif bir kalıp değildir. Madem zahmet edip kurulacak, özgün hâliyle kurmak daha mantıklıdır.

Bir gözleyici birden fazla nesneyi gözleyebilir; yapı buna izin verir. Gözlenen nesnenin durumundaki farklı türden değişikliklerin farklı türden gözleyicilerce izlenmesi istenebilir. Tutarlılık denetimleri yapılmak isteniyorsa onlar koda, örneğin `Attach` metoduna eklenmelidir.

8.3 Bildirimin tetiklenmesi

Hoca, pull-push seçimi kadar önemli bir kriter daha olduğunu söyledi: `Notify` mesajı nasıl ve kim tarafından tetiklenecek?

İlk akla gelen, gözlenen nesnenin tetiklemesidir. Subject'in durumu değişir değişmez `Notify` tetiklenmelidir. Somut subject'te `SetState` ya da durumun ayrı bileşenleri için ayrı setter'lar varsa, bu metotların son komutu olarak mutlaka `Notify` çağrılmalıdır.

> **Hocanın vurgusu:** Bildirimin tetiklenmesi subject tarafından yapılacaksa, `SetState` metodunun ve durumu değiştirebilecek bütün metotların sonunda `Notify` çağrısının kodlanması bir zorunluluktur.

Ama durum bilgisi çok sık değişiyorsa, hatta son değişiklik nesneyi bir iki önceki duruma geri çekebiliyorsa, her değişiklikte `Notify` tekrar tekrar çağrılır. Bu bir ağ üzerinden veri iletimiyse subject'ten observer'lara sürekli yoğun bir trafik olur. Pull modelde bu masraf biraz daha azdır; push modelde zaten çok olan masraf katlanır. Observer'lar dakikalık anlık değişikliklerle ilgilenmiyorsa bu aynı zamanda gereksiz trafiktir. Ayrı ayrı gidecek mesajları biriktirip toplu göndermek mümkündür, ama bu durumda subject'in `SetState` ya da `Notify` metodu ister istemez karmaşıklaşır. Bu önemli bir dezavantajdır.

Hoca önce gözleyicilerden söz etmeye başladı, sonra yanlış bir şey söylediğini belirtip şema üzerinden düzeltti. İkinci seçenek istemcidir (client). Somut subject'in durumunu değiştirebilecek bir istemci mutlaka olacaktır; `Notify` çağrısı istenen durum değişikliklerinin ardından bu istemciye kodlanabilir. Ama istemciyi kodlayan kişi bunu unutabilir; unutursa gözleyiciler durum değişikliklerinden haberdar olmaz. Bu önemli bir sorundur.

Üçüncü seçenek gözleyicilerin tetiklemesidir. Ama gözleyici değişikliğin ne zaman olduğunu bilmez. Belki bir zaman periyoduyla, örneğin 10 dakikada ya da 1 dakikada bir, gözlediği her şeyi sorgulayacak biçimde kodlanabilir. Sabit aralıklarla yapılan işler verimsiz olmaya eğilimlidir: değişiklik olmasa da boşuna haberleşme masrafıyla sorgu yapılır.

Sonuç olarak haberleşme masrafında optimizasyon yapılmak isteniyorsa en uygun yer yine gözlenen nesnedir. Bu, işi biraz karıştırır; ama elde edilecek kâr önemliyse bu karmaşıklık göze alınmalıdır.

8.4 Ne zaman kullanılır?

- Bir soyutlamanın birbirine bağımlı birden fazla parçası varsa ve bu bağımlılığın tutarlılığı korunmak isteniyorsa.
- Bir nesnedeki değişikliklerin, kodlama anında bilinemeyecek sayıda ve türde nesneyi etkilemesi gerekiyorsa.
- Bir nesnenin, ek bağlaşım koymadan diğer nesneleri bir durumdan haberdar etmesi gerekiyorsa. Farklı tür nesnelerin her biriyle ayrı bağlantı kurmaya gerek yoktur. Soyut Observer üzerinden yapılan çağrılarla herhangi bir somut observer ile iletişim kurulur ve subject hangi tür observer ile konuştuğunu bilmek zorunda kalmaz.

9. Observer Örneği: Müşteri İlişkileri Programı

9.1 Problem ve ilk çözüm

Hoca örneğin kaynağını hatırlayamadı; şemanın kendi çizimi olmadığını söyledi. Çözülecek problem şöyledir (ekrandan, 02:32:30): yazılan bir müşteri ilişkileri programına yeni müşteri eklendiğinde müşteriye bir "hoş geldiniz" e-postası gönderilecek ve müşterinin ikamet adresinin doğru olup olmadığına bakılacak.

Çözüm 1'de bu davranışlar sabit bir biçimde kodlanır:

```
Customer                - - - - >  WelcomeLetter   (generates Welcome Letters for customers)
+addCustomer()          - - - - >  AddrVerification (verifies address of customers)

Not: addCustomer calls WelcomeLetter and AddrVerification
```

`addCustomer` metodu hoş geldiniz mektubu için `WelcomeLetter` sınıfından bir nesneyi kendi içinde oluşturup ya da bir fabrikadan alıp kullanır. Adres doğrulamayı da benzer biçimde, belki bu amaçla hazırlanmış bir sınıfın statik metoduyla, belki bir bilgi sisteminden ya da bir fabrika metodundan aldığı nesneyle yapar. Her birinin ilgili metodu çağrılarak işler yaptırılır. Bu, akla gelen en basit, sabit kodlama biçimidir. Hoca, bu metodu `Customer` sınıfına koymanın da tam doğru olmadığını söyledi. Müşteri nesnesi nihayetinde bir bilgi sisteminde tutulacağı için metot aslında o bilgi sistemini simgeleyen başka bir sınıfa konmalıdır. Ama müşteriyle ilgili epey işlem olduğu için örnekte oraya kodlanmıştır.

9.2 Gereksinimler değişince

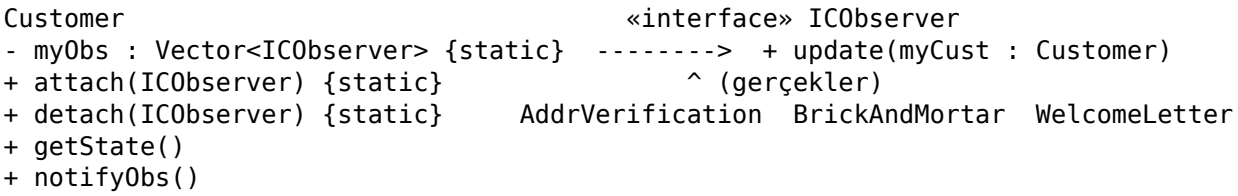
Slayttaki soru: "Peki ya gereksinimler değişirse?" Yeni bir istek gelir: şirketin çeşitli mağazaları ve şubeleri vardır. Müşteri bir mağazanın 10 km yarıçaplı dairesi içindeyse ona indirim kuponlu bir mektup gönderilsin; gelsin, alışveriş yapsın. Gözü açılmış bir programcı bu kampanyaların sonunun gelmeyeceğini anlar. Bu yüzden şimdiden altyapı kurulur. Hem sisteme her yeni müşteri eklendiğinde bir ya da birkaç farklı işlemin yapılabilmesi sağlanmalı, hem de müşteri eklenince sistem bizi haberdar etmelidir. Bu olay bir şeyi tetiklesin ve biz de ona göre iş yapalım diye düşününce akla Observer kalıbı gelir.

Hoca öğrencilere başka bir kalıp akıllarına gelip gelmediğini sordu ve Command'ı kendisi tartıştı. Command'da komut gönderilir ve benzer bir örnek de yapılmıştı; belki o da olabilir. Ama kritik fark şudur: "yeni müşteri eklendi" komutunu elle vermek başka, şu an gelen istek başkadır. İstenen, yeni bir müşteri eklendiğinde bu işlerin yapılması ve başka işlemlerin de eklenebilmesidir. İleride "yeni bir şube eklendiğinde" gibi istekler de gelebilir. Bir durum değişikliğinde sistemin haberdar edilip ilgili işlemlerin otomatik yapılması söz konusu olduğu için problem Observer kalıbıyla ilişkilendirilir.

Sonraki şemada "brick and mortar" diye bir terim geçer. Bir kurum hem sanal hem fiziksel ticaret yapıyorsa, fiziksel binalarda yaptığı ticarete İngilizce literatürde brick and mortar denir; brick tuğla, mortar harç demektir.

9.3 Observer ile çözüm

Çözüm 2'nin şeması şöyledir (ekrandan, 02:40:00):



Not: notifyObs metodu gözleyicilerin update metodunu çağırarak onları haberdar eder.

Slayttaki ek madde: yalnız müşteri nesneleri izlenecekse, bir soyut Subject üst sınıfı hazırlamaya gerek kalmadan Observer kalıbı kullanılabilir.

Şu an odak noktası müşteridir, ama bu yapı yeni bir şube gibi başka varlıklara da uyarlanabilir; onların her biri birer subject gibi davranır. Arayüzün adı `ICObserver`'dır: I arayüzü (interface), C müşteriyi (Customer) gösterir; yani müşterilerdeki değişiklikleri dinleyen gözleyiciler. Adres doğrulama, yeni açılan şubenin 10 km yakınındaki müşterilere özel indirim kuponu veren `BrickAndMortar` ve hoş geldiniz mektubu bu arayüzü gerçekler.

Hoca arayüzü yalnız `CustomerObserver` diye kısıtlamanın doğru olup olmadığını da tartıştı. Müşteriden sonra şube (location) gibi başka şeylerin de dinlenmesi gerekebilir. Bunlar için ayrı arayüzler tanımlanırsa sorun kalmaz. Örneğin adres doğrulama onlar için de gerekiyorsa `AddrVerification` bu arayüzlerin hepsini gerçekleyebilir; modern diller bir sınıfın birden fazla arayüz gerçeklemesini sorunsuz destekler. Belli türden bir observer'ın belli türden subject'leri dinlemek istemesi son derece mantıklı bir senaryodur ve bu örnek ona göre düşünülmüştür.

Kalıpta normalde soyut Subject ve soyut Observer sınıfları vardır. Bu örnekte yalnız müşteri nesneleri izleneceği için ayrıca bir soyut Subject sınıfı konmadı; `attach` ve `detach` doğrudan `Customer`'dadır. Bütün müşteri nesneleri hep aynı gözleyicilerce izlenecekse gözleyici listesi statik kodlanabilir. Liste statik olunca `attach` ve `detach` de statik olmak zorundadır. Kalıp bunların statik olmasını kesinlikle gerektirmez; bu örneğe özgüdür.

Location gibi başka subject sınıfları da olacaksa soyut Subject sınıfını eklemek yararlıdır. Aksi hâlde gözleyici üye alanı ile `attach`/`detach` işleri o sınıfların her birinde yinelenen kod (duplicate code) olarak yazılır. Böyle sınıflar söz konusuysa çözüme soyut Subject mutlaka eklenmelidir.

9.4 Kod

`ICObserver` arayüzü `update` metodunu içerir. `Customer` sınıfı şöyledir (ekrandan, 02:42:15):

```
package dp.observer.example;
import java.util.*;

public class Customer {
    private static Vector<ICObserver> myObs =
        new Vector<ICObserver>( ); //Gözleyiciler ortak olacak.
    public static void attach(ICObserver o){
        myObs.addElement(o);
    }
    public static void detach(ICObserver o){
        myObs.remove(o);
    }
    public String getState () {
        // TODO: Burada durum bilgisini döndür
        return null;
    }
    public void notifyObs () {
        for( ICObserver obs : myObs ) {
            obs.update(this);
        }
    }
}
```

Kodun bir kısmı aslında subject işleridir: gözleyicilerin tutulması, `attach` ve `detach`. Tek subject türü müşteri olduğu için bunlar ayrı bir sınıfa alınmadı; gerekirse sonra ayrı bir sınıfa çıkarılır (extract). `getState()` şimdilik iskelet koddur. `notifyObs()` bütün gözleyicilerin `update` metodunu çağırır ve nesnenin kendisini verir.

Hoca kodu incelerken pull mu push mu modelle çalışıldığını sordu. `update` parametre olarak müşteri nesnesinin tamamını aldığı için subject'in (müşterinin) bütün durumu gönderilir; demek ki push modelle çalışılmıştır.

`AddrVerification`, `BrickAndMortar` ve `WelcomeLetter` sınıflarının `update` metotları parametre olarak gelen müşterinin durumunu alır. Ne yapılması gerekiyorsa (yazmak, bir yerden sorgulamak) yapılacak yerler "TODO" notuyla iskelet kod olarak bırakılmıştır.

10. Birbirine Benzeyen Kalıplar Üzerine

Dersin sonunda bir öğrenci, bazı kalıpların birbirine çok benzediğini, kullanılış biçimleri farklı olsa da birbirinin yerine kullanılabildiğini söyledi. Bir problemin hangi kalıpla çözüleceği sorulduğunda tek bir doğru cevap olup olmadığını sordu.

Soru: Bir problem için birden fazla kalıp uygun olabilir mi; tek bir doğru cevap var mıdır?
Cevap: Hoca birden fazla kalıbın birbirinin yerine kullanılabileceğini söyledi. Böyle bir problemde önemli olan, kalıbın adıyla birlikte neden o kalıbın seçildiğinin açıklanması ve tasarımın kalıpla bir UML sınıf şeması üzerinde gösterilmesidir. Hocanın aklında genelde bir ya da iki kalıp olur; ama farklı bir kalıp önerilip açıklaması ikna ediciyse o da doğru kabul edilir. Seçilen kalıbın sağladığı yapı problemin isterlerinin yalnız bir kısmını karşılıyorsa, karşılanmayan kısımlar için ek bir kalıp önerilmelidir; aksi hâlde çözüm eksik kalır.

Hoca ayrıca, hem kullanım amacı hem de sınıf şeması açısından birbirine benzeyen kalıplar olduğunu tekrarladı. Fark ayrıntılardadır: hangi metotların ne sıklıkla çağrıldığı ve kalıbın hitap ettiği çekirdek problemin farklı olması gibi. Bunu dersin ilk haftalarında da söylemişti. Daha somut örneğini Strategy ve Builder kalıplarından sonra tartışacaklarını belirtti.

11. Örnek Sorular

Soru 1

(derste sözlü, 00:10:59) Kökün iki çocuğu, onların da ikişer çocuğu olan tam bir ikili ağaçta kökün değeri 9, kökün sol çocuğu 5, onun sol çocuğu 3 ve sağ çocuğu 6 olsun. LDR (Left-Data-Right) dolaşımında elemanlar hangi sırayla ele alınır?

> **Çözüm (derste verildi):** Kökten başlanır ve sol işaretçiler izlenerek en sola, 3'ün bulunduğu düğüme inilir. Solu olmadığı için 3 alınır; sağı olmadığı için bir üste dönülür ve 5 alınır. 5'in sağ çocuğuna gidilir ve 6 alınır. Köke dönülüp 9 alınır, sonra kökün sağ alt ağacı aynı kuralla dolaşılır. Sol alt ağaç ve kök için sıra 3, 5, 6, 9'dur. Sağ alt ağacın değerleri derste verilmedi.

Soru 2

(derste sözlü, 00:51:18) Iterator ile yazılan `Waitress` sınıfında `printMenu` ile `printVegetarianMenu` arasındaki fark nedir ve bu tekrar nasıl giderilebilir?

> **Çözüm (derste verildi):** Tek fark, iteratordan `next` ile eleman alındıktan sonra gelen `if (vejetaryense)` denetimidir. Iterator oluşturulurken ona "bütün üyelerde dolaş / vejetaryen üyelerde dolaş" diye bir yapılandırma verilirse fark ortadan kalkar. Ama hazır `java.util.Iterator`'a metot eklenemez; kendi iterator sınıfını yazmak gerekir. Bu yatırım yalnız vejetaryen seçeneği için değmez; glutensiz, çocuk menüsü gibi seçenekler çoğalacaksa yapılmalıdır.

Soru 3

(derste sözlü, 01:36:40) Java'daki iç sınıflı Memento çözümünde `State.clone()` kopya oluşturmak yerine `return this;` döndürürse `Caretaker.test()` ne yazdırır?

Çözüm (derste verildi): Derste denendi: `setMemento(mem)` çağrılmasına rağmen çıktı "yeni durum" olur, yani ilk duruma geri dönülemez. Kopya döndürüldüğünde çıktı "ilk durum"dur.

Soru 4

(derste sözlü, 01:38:32 ve ekrandan, 01:40:00) İç sınıflı Memento çözümünde Memento'nun bütünlüğü garanti altında mıdır? `testBroken()` metodundaki deneme neyi gösterir ve nasıl önlenebilir?

Çözüm (derste verildi): Caretaker Memento'nun yalnız `Object` metotlarına erişebilir; Memento'yu doğrudan değiştiremez. Ama `testBroken()`'da ikinci bir Originator (`fakeSubject`) "sahte durum"a getirilip onun Memento'su asıl nesneye verildiğinde çıktı "sahte durum" olur. Yani dolaylı yoldan durum değiştirilebilir. Önlem: Originator'a kurucuda rastgele ilklendirilen, getter'ı olmayan private final bir kimlik konur; Memento bu kimliği taşır ve `setMemento` kimlik eşleşmezse durumu atamaz.

Soru 5

(derste sözlü, 02:34:27) Müşteri ilişkileri problemi için Observer dışında hangi kalıp akla gelebilir ve neden Observer tercih edilir?

Çözüm (derste verildi): Command akla gelebilir; derste benzer bir örnek yapılmıştı. Ama Command'da komut elle verilir. Bu problemde istenen, bir durum değişikliğinde (yeni müşteri eklenmesi) sistemin haberdar olup ilgili işlemleri otomatik yapmasıdır; ileride başka olaylar ve işlemler de eklenebilir. Bu yüzden Observer daha uygundur.

Soru 6

(derste sözlü, 02:41:08) Müşteri örneğinde (`ICObserver.update(myCust : Customer)`, `notifyObs()` içinde `obs.update(this)`) pull model mi, push model mi kullanılmıştır?

Çözüm (derste verildi): `update` metodu müşteri nesnesinin tamamını parametre olarak aldığı için subject'in bütün durumu gözleyicilere gönderilir. Bu yüzden örnek push modelle çalışır.

Soru 7

(derste sözlü, 02:37:05) Gözleyici arayüzünü yalnız müşteri gözleyicisi (`ICObserver`) olarak kısıtlamak doğru mudur?

Çözüm (derste verildi): Sorun olmaz. Şube gibi başka subject türleri gelirse onlar için ayrı gözleyici arayüzleri tanımlanır ve bir sınıf (örneğin `AddrVerification`) birden fazla arayüzü gerçekleyebilir. Ama birden fazla subject türü olacaksa `attach/detach` ve gözleyici listesinin tekrarlanmaması için soyut bir Subject sınıfı eklenmelidir.

Dersin Toparlaması

Bu derste davranışsal kalıplardan üçü işlendi. Iterator, bir bütünün parçalarını iç yapısını açmadan dolaşmayı sağlar. Bunu dolaşım sorumluluğunu ayrı bir sınıfa alarak ve somut iteratorları bir fabrika metoduyla ürettirerek yapar. Menü örneği, iki farklı veri yapısının ortak bir arayüz arkasında eş biçimli dolaşılmasını ve bunun yinelenen kodu nasıl azalttığını gösterdi. Memento, durumun sarmalamayı bozmadan saklanıp geri yüklenmesini sağlar. Java'da iç sınıf ve durumun klonlanmasıyla gerçeklenmesi denendi; bu çözümün de başka bir nesnenin Memento'sunun verilmesine karşı ek bir kimlik denetimine ihtiyaç duyduğu görüldü. Observer, bir nesnedeki değişiklikten ilgili nesnelerin gevşek bağlaşımla haberdar edilmesini sağlar. Pull/push seçimi ve bildirimin kim tarafından tetikleneceği, kalıbın en önemli gerçekleme kararlarıdır.

Hoca birbirine benzeyen kalıpların hitap ettikleri çekirdek problem ve metot çağrı sıklığı gibi ayrıntılarla ayrıldığını hatırlattı. Bu karşılaştırmanın somut örneğinin Strategy ve Builder kalıplarından sonra yapılacağını söyledi. Sırada Strategy, State, Visitor ve Mediator gibi kalıplar vardır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.